Recurrent Neural Networks (RNN):

RNNs process sequences step-by-step, maintaining a hidden state (memory).

“Read one word → update memory → move to next word”

Strengths of RNN

- Captures **temporal dependencies**
- Works well for:
    - Time series (stock, weather)
    - NLP (basic tasks)
    - Speech recognition

Limitations:

Sequential Processing

- Cannot parallelize → slow training

Short-Term Memory Bias

- Struggles with long-range dependencies

RNN is a neural network that processes sequences step-by-step using a shared memory (hidden state), but struggles with long dependencies and parallelization.

Attention Mechanism:

It was designed mainly for sequence-to-sequence tasks, like:

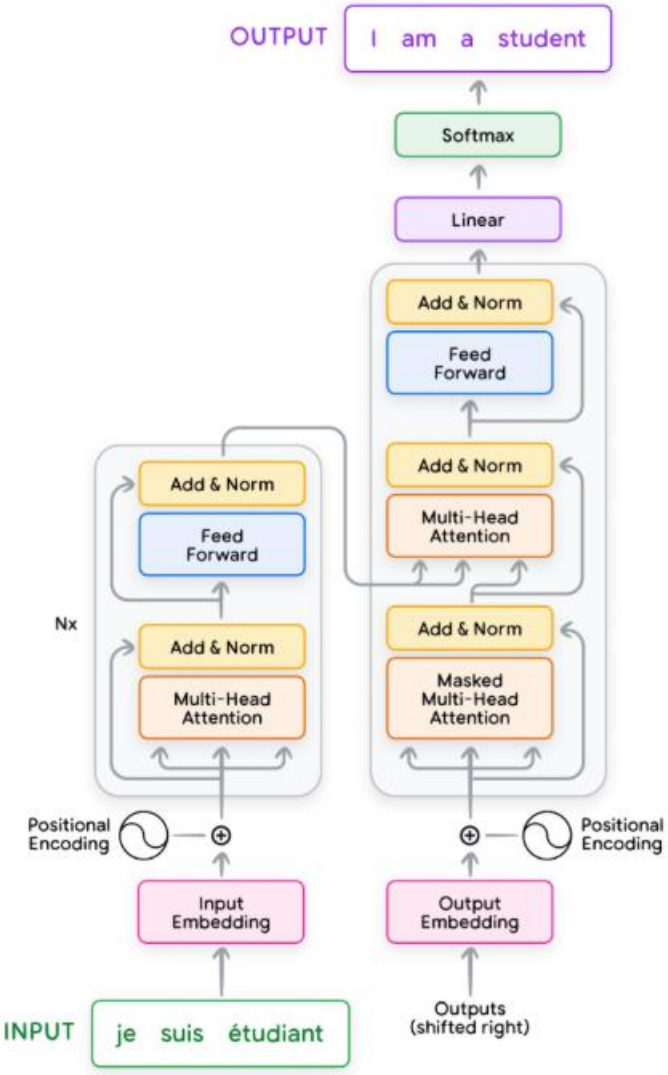
- language translation
- text summarization
- question answering

Transformer was improved in

- removing recurrence.
- processing words in parallel.
- using **attention** to understand relationships between words.

A Transformer has 2 main parts:

Encoder:
Reads and understands the input sentence.
Decoder:
Generates the output sentence one word at a time.
So architecture is:
Input sentence → Encoder → Decoder → Output sentence



Idea of self-attention:

Instead of reading the sentence one word at a time, Transformer looks at all words together.

For each word, it checks:

- Which other words are important?
- How much should focus need on them?

Overall architecture:

Encoder stack

Usually multiple encoders, one after another

1. Multi-Head Self-Attention

Each word looks at all other words in the sentence and decides which words are important

Create Query (Q), Key (K), Value (V)

Compute attention scores:

Instead of one attention:

- Multiple attention heads learn different relationships:
  - Head 1 → grammar
  - Head 2 → meaning

- Head 3 → position/context

2. Add & Layer Normalization

- Residual = "don't forget original meaning"
- Normalization = "keep values balanced"

3. Feed Forward Network (FFN)

Applies a small neural network independently to each word

- Attention mixes information across words
- FFN processes each word deeply & think about it.
- transformation within each word

4. Add & Layer Normalization (after FFN)

- Stabilize and preserve learning

Decoder:

Each decoder layer has:

1. Masked multi-head self-attention
2. Add & Layer Normalization
3. Encoder-decoder attention
4. Add & Layer Normalization
5. Feed Forward Network
6. Add & Layer Normalization

Input sentence is tokenize:

1. Normalization (Optional): Standardizes text by removing redundant whitespace,

accents, etc.

2. Tokenization: Breaks the sentence into words or subwords and maps them to integer(index) token IDs from a vocabulary.

3. Embedding: Converts each token ID to its corresponding high-dimensional vector, typically using a lookup table. These can be learned during the training process. embeddings do not contain order.

4. Positional Encoding: Adds information about the position of each token in the sequence to help the transformer understand word order. embeddings do not contain order.

This helps the model know:

- which word is first
- which is second
- relative positions

Creating queries, keys, and values: Each input embedding is multiplied by three learned

weight matrices (Wq, Wk, Wv) to generate query (Q), key (K), and value (V) vectors. These

are like specialized representations of each word.

• Query: The query vector helps the model ask, “Which other words in the sequence are relevant”

• Key: The key vector is like a label that helps the model identify how a word might be relevant to other words in the sequence.

• Value: The value vector holds the actual word content information.

Ex : “The cat eats fish”
Query of “eats” asks: which words matter for me?
Keys of all words say: this is what I represent
Values contain their actual information

This gives attention scores.

Attention score calculation:
We compare Query of one word with Keys of all words.
Using dot product:

$$\text{score} = QK^T$$

This gives similarity between words.
If similarity is high:

- that word is important

If similarity is low:

- that word is less important

Scaling:
Where d_k is dimension of key vectors.
So formula becomes:

$$\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}$$

This stabilizes training.

Softmax:
Now convert scores into probabilities:

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

This ensures:

- all weights are between 0 and 1
- all weights sum to 1

Weighted sum of values

- Now multiply attention weights with Value vectors:

- $\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
  This gives a new contextual representation for each word.
- So now “eats” is no longer just its original embedding.
  It becomes a representation that includes:
- subject information from “cat”
- object information from “fish”
- This is the heart of Transformer.

Masking:
In Transformers, masking is a mechanism used to control what information a token is allowed to “see” during attention computation.
It is critical for correctness—especially in the decoder.
A mask in Transformers is used to block certain positions during attention computation. The look-ahead mask prevents the decoder from accessing future tokens during training, ensuring autoregressive behavior, while the padding mask prevents the model from attending to padded positions in variable-length sequences.

Look-ahead mask
A **look-ahead mask** blocks the decoder from seeing **future words**.
When generating a sentence, the model should predict the next word using only:
- previous words
- current position
- It should **not** see words that come later.

Autoregressive behavior:
Generate one token at a time, using previously generated tokens to predict the next one.
To predict the next word, the model depends on earlier words.

Padding mask:
A padding mask hides padding tokens.
Sentences in a batch may have different lengths.
PAD is just a dummy token added to make equal length as of previous sentence.

Suppose we have 3 sentences:
- “The cat eats fish” → length 4
- “Dogs run” → length 2
- “I like deep learning” → length 4

After tokenization:
- [The, cat, eats, fish]
- [Dogs, run]
- [I, like, deep, learning]

A model does not take these as loose lists. It takes them as a **matrix / tensor**.

Large Language Model (LLM):

A Transformer-based neural network trained on massive text corpora to model the probability distribution of token sequences.

Fine tuning in LLM:

Fine-tuning in an LLM means taking a pretrained model and training it a bit more on a specific task, behavior, or domain so it performs better for that use case.

A pretrained LLM is trained on broad internet-scale data.
It is good at:

- general language
- general reasoning patterns
- common knowledge patterns

But it may not be good enough for:

- legal domain
- insurance domain
- finance domain

Pretraining teaches the model:
"How language works"
Fine-tuning teaches the model:
"How to behave for this specific task"

- **pretraining** = broad knowledge
- **fine-tuning** = specialization

Types of fine-tuning

A. Full fine-tuning

Every weight in the model can change.

B. Instruction fine-tuning

The model is trained on instruction-response pairs**.**

Teach the model to follow human instructions properly.

C. Supervised Fine-Tuning (SFT)

This means training on labeled examples where a "correct" response is provided. This is usually the first alignment step after pretraining.

D. Parameter-Efficient Fine-Tuning (PEFT)

Instead of updating all weights, update only a small part.
This is very important in real industry systems.
Modern LLMs are huge.
Full fine-tuning is expensive.

So PEFT methods save:

- memory
- time
- storage

D1. LoRA (Low-Rank Adaptation)

Instead of changing the full weight matrix W, LoRA adds a small trainable update:
Do not retrain the giant matrix.
Only learn a small correction.

Pros

- much cheaper than full fine-tuning
- very popular in practice
- easy to maintain multiple task adapters

D2. Adapters:

Small trainable layers are inserted inside the Transformer blocks.
Base model is frozen, adapters are trained.
Pros

- lightweight
- modular

D3. Prompt tuning / Prefix tuning

Train a small set of learnable prompt embeddings instead of updating the whole model.
Model remains frozen, but learned prompts guide behavior.

RLHF = Reinforcement Learning from Human Feedback

Humans compare answers and provide preferences.
Then the model is optimized to produce preferred responses.
This is more about alignment than ordinary domain fine-tuning, but it is part of the broader adaptation pipeline.

- **Fine-tuning vs prompt engineering:**
  These are not the same.
  Prompt engineering

You do not change model weights.
You only change the input prompt.
Example:
"Answer like an insurance expert in simple language."

Fine-tuning

You actually update model parameters.

- **Fine-tuning vs RAG:**
  Fine-tuning
  Changes model behavior/weights.

Fine-tuning is not ideal for constantly changing documents.

RAG (Retrieval-Augmented Generation)
Does not mainly change weights.
Instead, it retrieves relevant documents and gives them to the model at runtime.

Best for:

- up-to-date information
- company documents
- knowledge bases
- factual grounding